Bar feeds, the cache and the borrow check

Where qp's price bars come from, how history is extended for indicator warm-up, how the bar cache is kept from filling the disk, and the short-borrow check. Written from `quant-platform/chart/data_manager.py`, `tools/data/*.py`, `tools/compare_server.py` and the feed routes in `chart/server.py`.

Related: the full data check, which judges every feed's content (DATACHECK.md); the CANSLIM sources (CANSLIM.md).

One place for bars

The chart, the compare tool, strategies, backtests and alerts all fetch through the same loaders (`tools/data/`), each returning the same shape: a time-zone-aware (UTC) frame of open, high, low, close, volume.

Feed	Source	Strength	Weakness
`yahoo`	Yahoo Finance	free, current, years of daily history	shallow 1-minute history
`yahoo_ext`	Yahoo with its own pre-market and after-hours	one source, more hours: for strategies that read pre-market levels	—
`polygon`	Polygon.io	deep history, full extended hours, consolidated tape	the free plan is end of day: today's bars missing
`alpaca`	Alpaca IEX	live	IEX only: thin volume, about a week of intraday history, regular hours
`hybrid`	Polygon history + Alpaca for today	deep history and today	**volume seam**: Alpaca IEX volume is a fraction of Polygon's, which distorts every VWAP on the Alpaca part
`hybrid_yahoo`	Polygon history + Yahoo for today	deep history and today, both consolidated	—

The default feed is set by hand (More → Feed → *make default*, `POST /api/settings/default-feed`), saved in `quant-platform/.default-feed`, and used by every chart and backtest that does not name one. It used to be inferred (Polygon whenever a key existed), and a key being

present turned out not to mean the plan behind it covered the data: every 1-minute request was refused while the platform called Polygon its best feed. Without a saved choice the default is `yahoo`.

Some routes still name `polygon` as their own default when no feed is passed (`/api/chart`, `/api/strategy/explain`, the print sheets); the page always passes one (Q-N6).

Keys come from the environment or `quant-platform/.env`: the deploy copies the desk's Alpaca pair there from `data/keys.json` (`scripts/sync-qp-env.js`); `POLYGON_API_KEY` is optional.

Warm-up and the live tail (`chart/data_manager.py`)

- **Auto warm-up:** the fetch window is extended so every indicator on the chart or in the strategy has enough history before the first bar shown. No manual juggling of days.
- **Live tail:** with ▶ **Live**, the server re-computes on an interval and pushes only the newest bar and each indicator's newest value over `/ws/live`.
- **As of a day:** `asof` hides every bar after that date, for replay and for the backtest.

Which feeds actually answer

Check	Asks	Use
`GET /api/health`	which feeds have credentials	an inventory only: a present key is not a working one
`GET /api/feedcheck`	one small real fetch per feed, judged by the data check's own rules	the deploy prints this. It replaced a line that said "alpaca ok" every day while the Alpaca key was being refused with a 401
`GET /api/datacheck`	every loader, both indexes, cross-source agreement, the RS universe, the files qp builds, EDGAR	the full, slow answer (DATACHECK.md)

`feedcheck` never raises and never answers 4xx: its caller is a deploy script, where an exception would read as "the feed is down".

The bar cache (`tools/data/cache.py`)

Bars are cached as parquet files under `~/.qp-cache/` (`QP_CACHE_DIR`), per feed (Polygon files are prefixed so feeds never mix).

Why it has limits: nothing used to delete a cached file. A print sheet is up to 150 charts, each a multi-day 1-minute frame; a few afternoons of printing is gigabytes, and the day the disk filled, the platform (and every program on the box) stopped.

Trigger	Action
the cache is over its ceiling (`QP_CACHE_MAX_MB`, default **512 MB**)	trim the oldest files down to 70 % of the ceiling
the **disk** has less than `QP_CACHE_MIN_FREE_MB` free (default **1024 MB**)	trim hard, whatever the cache's size: the target is computed from what is there, not from the ceiling

It trims where it grows: every loader calls the sweeper after writing. On the page (More → Bar cache) and through the API:

Route	Does
`GET /api/cache`	size, free disk, whether it is over
`POST /api/cache/clear`	delete everything (`older_than_days` deletes only older files); safe, because every file can be fetched again
`POST /api/cache/sweep`	trim to the limit now

The CANSLIM caches (`grouped/`, `sic/`, `f13/`, `canslim/`, EDGAR facts) are documented in CANSLIM.md and DATA-MODEL.md.

The borrow check (`tools/data/borrow.py`)

Will the broker let this account short the name? Backtest #354 made +$1,345 across twelve trades, and one short (XE) made +$1,409 of it; Alpaca answers that XE cannot be sold short, so without it the run lost $63. Most of what the screeners find is a small cap with no borrow, so for a short book this is the common case.

- Alpaca reports `shortable` **as it is now**; there is no history. A backtest applies **today's** flag to a past day and labels it as such.
- **An unanswerable check is not a refusal:** if Alpaca cannot be reached the answer is "unchecked" and the trade stands, because dropping trades over a timeout would turn a network blip into a strategy result.
- The backtest report says loudly when the check was asked for and nothing was answered (BACKTEST.md).